

User Study

Instructions for Grading. **PLEASE READ CAREFULLY.**

Overview

Imagine you are a user interacting with a chatbot service (e.g., Gemini, ChatGPT, or DeepSeek) for help with programming tasks. On each page, you will see:

- **A Question:** This could be a vague user query or a code snippet requiring clarification or completion. Questions containing code will be displayed in code blocks for clarity.
- **Two Responses:** labeled Answer A and Answer B

Your Task

Please evaluate Answer A and Answer B based on the following criteria:

- **Precision and Focus:** How directly the response targets the question's core issue, using clear guidance, examples, or code.
- **Contextual Fit:** How well the response matches the user's programming goal (e.g., debugging, feature implementation).
- **Answer Faithfulness:** How effectively the response provides actionable steps or code to achieve the user's desired outcome.
- **Correctness:** Correctness in LLM responses refers to the accuracy and factual validity of the information, code, or guidance provided, ensuring it is free from errors and aligns with the user's query and intended outcome.

How to Rate

For each question, you will provide two types of ratings:

1. **Preference Rating (1–5 scale):**
 - **"1"**: Answer A is better, based on the criteria above.
 - **"5"**: Answer B is better, based on the criteria above.
 - **"3"**: Both answers are equally good, based on the criteria above.

Note:

- **Incomplete Responses:** Some responses may appear cut off due to a 256-word limit imposed on the chatbots. Please grade only the content provided, ignoring any abrupt endings or non-English text (e.g., stray characters).
- **Code Formatting:** Questions and responses containing code will be presented in code blocks or artifact tags for clarity. Ensure you review these carefully for correctness and relevance.
- **Consistency:** Try to apply the same standards across all evaluations to ensure fair comparisons.

Question 1

JavaScript

```
var myArray = [];  
  
function addElement(element) {  
    myArray.push(element);  
    // TODO: handle duplicates  
}
```

Answer A:

JavaScript

```
var myArray = [];  
  
function addElement(element) {  
    if (myArray.indexOf(element) !== -1) {  
        console.log("Element already exists in myArray."); //  
Optional: Log the duplicate  
        return; // Prevent adding the duplicate  
    }  
    myArray.push(element);  
    // TODO: handle duplicates  
}  
  
// Example usage:  
addElement("apple");  
addElement("banana");  
addElement("apple"); // This will log "Element already exists in  
myArray."  
addElement("orange");
```

Explanation:

1. `var myArray = [];`: Initializes an empty array.
2. `function addElement(element) {...}`: Defines a function to add elements.
3. `if (myArray.indexOf(element) !== -1) {...}`: Checks for duplicates before adding.

Answer B:

“Extra context:

User Question:

JavaScript

```
var myArray = [];  
  
function addElement(element) {  
    myArray.push(element);  
    // TODO: handle duplicates  
}
```

AI clarification question: What is the intended use case for this `addElement` function? (e.g., is it intended to add elements to a list, a set, or something else?)

User follow up response: I want to prevent adding duplicate elements to the array.”

AI Final answer:

You're absolutely right to question the intended use case! The code snippet is a very basic placeholder for adding elements to an array. The comment "TODO: handle duplicates" is crucial. The intended use case is to prevent duplicate elements from being added to `myArray`. Currently, the code simply pushes each element into the array without checking if it already exists. This will result in duplicate elements being added, which is generally undesirable.

- `var myArray = [];`: Initializes an empty array.
- `function addElement(element) {...}`: Defines a function to add elements.
- `myArray.push(element);`: Adds the element to the array, but does not check for duplicates.

To prevent duplicates, you should check if the element already exists before adding it.

How to rate:

"1": strongly prefer answer A

"5": strongly prefer answer B

"3": both answers are equally good

Precision and Focus (1-5): 2

Contextual Fit (1-5): 3

Answer Faithfulness (1-5): 2

Correctness (1-5): 3

Question 2

Java

```
public class Sample {  
    private int number;  
  
    // TODO: Add constructor  
}  
  
Sample s = new Sample();
```

Answer A:

"Extra context:

User Question:

Java

```
public class Sample {  
    private int number;  
  
    // TODO: Add constructor  
}  
  
Sample s = new Sample();
```

AI clarification question: What is the intended purpose or functionality of the *Sample* class? Specifically, what kind of data should the class be designed to hold and how should it interact with the outside world?

User follow up response: The class should hold a number and initialize it via a constructor. Default initialize number to 3."

AI Final answer:

To add both a default constructor and a parameterized constructor in your *Sample* class, you can define two constructors. The default constructor will set *number* to a predefined value (e.g., 3, as you mentioned), and the parameterized constructor will accept a user-provided value.

Here's how to implement both constructors:

```
Java
public class Sample {
    private int number;

    // Default constructor
    public Sample() {
        this.number = 3; // Default initialize to 3
    }

    // Parameterized constructor
    public Sample(int number) {
        this.number = number; // Initialize with user-provided value
    }
}

// Example usage
class Main {
    public static void main(String[] args) {
        Sample s1 = new Sample(); // Uses default constructor
        Sample s2 = new Sample(10); // Uses parameterized constructor
    }
}
```

Explanation

1. **Default Constructor** (public Sample()):
Takes no parameters.
Initializes number to 3, as you specified.
Called when you create an object with new Sample().
2. **Parameterized Constructor** (public Sample(int number)):
Takes an int parameter.
Assigns the provided value to number.
Called when you create an object with new Sample(10) or any other integer.
3. Usage:
Sample s1 = new Sample() creates an object with number set to 3.
Sample s2 = new Sample(10) creates an object with number set to 10.
The getter getNumber() is included for testing to verify the initialized values.

Answer B:

Java

```
public class Sample {  
    private int number;  
  
    public Sample(int number) {  
        this.number = number;  
    }  
  
    public int getNumber() {  
        return number;  
    }  
}
```

Explanation:

- `public class Sample { ... }`: Defines the class.
- `private int number;`: Private field to store the number.
- `public Sample(int number) {...}`: Constructor to initialize the number.
- `public int getNumber() {...}`: Getter to access the value.

How to rate:

"1": strongly prefer answer A

"5": strongly prefer answer B

"3": both answers are equally good

Precision and Focus (1-5): 2

Contextual Fit (1-5): 2

Answer Faithfulness (1-5): 1

Correctness (1-5): 1

Question 3

There's an issue with my API integration.

Answer A:

Okay, I understand you're having an issue with your API integration. That's frustrating! To help me understand and potentially troubleshoot, I need a little more information. Let's break this down.

First, can you tell me:

1. What API are you trying to integrate with? (e.g., Google Maps, Twitter, Stripe, a specific internal service, etc.)
2. What exactly is the issue you're experiencing? Be as specific as possible.
 - Error Messages: Can you copy and paste the exact error message you're seeing?
 - What's Happening?: What are you expecting to happen, and what actually happens?
 - Steps to Reproduce: Can you walk me through the steps to make the problem happen again?
 - What you've already tried? Have you tried any troubleshooting steps already?

Answer B:

"Extra context:

***User Question:** There's an issue with my API integration.*

***AI clarification question:** Could you tell me what API you're using and what specific issue you're encountering? For example, are you getting errors, or is it just not working as expected?*

***User follow up response:** I'm integrating the Stripe API, and the payment processing fails with a 'invalid_request_error'."*

AI Final answer:

Thanks for the details! With Stripe, an `invalid_request_error` usually means something is wrong with the parameters you're sending.

Here's what you should check:

- Make sure all required parameters (like amount, currency, source, etc.) are being sent and have valid values.
- Double-check your API keys and environment (test vs. live).
- Review the Stripe API documentation for the endpoint you're calling.

- Check the full error message returned by Stripe for more specifics.

If you share your request code (with sensitive info removed), I can help you debug further!

How to rate:

"1": strongly prefer answer A

"5": strongly prefer answer B

"3": both answers are equally good

Precision and Focus (1-5): 3

Contextual Fit (1-5): 3

Answer Faithfulness (1-5): 3

Correctness (1-5): 3

Question 4

I'm confused about data structures.

Answer A:

Okay, let's tackle data structures! It's a really broad topic, and it's totally normal to feel confused. Don't worry, we'll break it down. Let's start with some foundational questions and then I'll guide you through some key concepts.

1. What is a Data Structure?
 - A way of organizing and storing data in a computer so that it can be used efficiently.
2. Why do we need Data Structures?
 - Efficiency, problem solving, memory management.
3. Let's Start with the Basics - Common Data Structures:
 - Arrays, linked lists, hash tables, trees, graphs, stacks, queues, etc.

Answer B:

"Extra context:

User Question: *I'm confused about data structures.*

AI clarification question: *Could you tell me what area of data structures you're finding confusing? For example, are you unsure about what data structure to use, how to implement one, or something else?*

User follow up response: *I'm confused about how to choose between a hash table and an array for storing key-value pairs."*

AI Final answer:

Great question! The main difference is:

- Arrays are best for ordered data and when you access elements by index.
- Hash tables (objects/maps in JS, dicts in Python) are best for key-value pairs, especially when you need fast lookups by key.

If you need to associate values with unique keys and look them up quickly, use a hash table. If you just need a list of values in order, use an array.

How to rate:

"1": strongly prefer answer A

"5": strongly prefer answer B

"3": both answers are equally good

Precision and Focus (1-5): 4

Contextual Fit (1-5): 4

Answer Faithfulness (1-5): 3

Correctness (1-5): 3

Question 5

I need help with a function.

Answer A:

"Extra context:

User Question: *I need help with a function.*

AI clarification question: *Could you please share the function you're working on and tell me what part you're struggling with specifically? For example, are you having trouble with performance, readability, or something else?*

User follow up response: *I'm working on a Python function to sort a list, but it's too slow for large inputs."*

AI Final answer:

Thanks for clarifying! If your sort function is slow, it might be using an inefficient algorithm like bubble sort. For large lists, you should use Python's built-in `sorted()` function or the `list.sort()` method, which use Timsort (very efficient).

Example:

Python

```
def sort_list(data):  
    return sorted(data)
```

This will be much faster for large inputs than a manual bubble sort.

Answer B:

I need help with a function.

Okay, I'd love to help! Please tell me about the function you need help with. To help me understand your needs, please provide as much detail as possible, including:

1. What is the function supposed to do?
2. What is the input to the function?
3. What is the output of the function?
4. Do you have any code already?
5. What's the specific problem you're encountering?
6. Are there any constraints or limitations?

How to rate:

"1": strongly prefer answer A

"5": strongly prefer answer B

"3": both answers are equally good

Precision and Focus (1-5): 3

Contextual Fit (1-5): 3

Answer Faithfulness (1-5): 2

Correctness (1-5): 3